

软件过程改进方案设计文档

软件过程改进方案设计文档

1. 文档概述

本文面向 Dyad 项目的软件过程改进。原过程已经覆盖“立项与需求、系统与架构设计、工程实现、验证与确认、发布与部署、配置与支持、项目管理、过程改进”等活动，本方案在这些活动中加入 AI 执行节点，让人负责目标、取舍和确认，让 AI 负责候选方案生成、上下文分析和自动化检查。Dyad 是本地运行的 AI 应用构建工具，采用 Electron、React、TypeScript 技术栈，主要功能是用自然语言生成应用原型，并支持界面、代码、交互和本地预览的持续修改。它的流程不适合加重审批和文档负担，更需要高频、易返工、依赖上下文的地方加一层 AI 辅助。

方案沿用前期报告识别出的 5 个改进环节：需求捕获与一致性验证、架构设计与 UML/ADR 生成、仓库级编码与代码审查、测试生成与自愈测试、发布运维异常检测与根因分析。每个环节都给出角色调整、活动流程、AI 输出物标准、验证方式、工具链选型、效率指标和风险控制。AI 输出只作为工程输入，进入评审或自动化质量门禁后才能继续流转。

2. 改进目标与原则

本次过程改进主要解决两个问题：一是把需求澄清、设计文档、代码实现、测试补充和发布问题定位的时间降下来；二是减少需求不清、架构边界模糊、测试不足和发布反馈滞后带来的返工。Dyad 仍然采用轻量过程，但每个 AI 参与过的工件都要留下来源、版本和验证记录。数据安全、模型幻觉、工具依赖和质量责任也要放进流程里处理，不能等出问题后再补规则。

过程改进遵循以下原则：

1. 人类保留最终责任。AI 可以生成需求、设计、代码、测试和诊断建议；需求基线、架构决策、代码合并、发布和回滚由负责人确认。
2. AI 输出要能验证。代码、测试、配置、发布脚本类输出走自动化检查；需求、设计、风险类输出走人工评审和清单检查。
3. 先从低风险高收益环节落地。需求整理、代码审查、测试生成和发布说明生成先做，再扩展到架构智能体和根因分析智能体。
4. 保持轻量过程。AI 节点嵌入现有 issue、PR、CI、测试和发布流程，不单独堆出一套审批链。
5. 建立可度量指标。每个改进环节都定义效率和质量指标，如需求整理时间、PR 审查时间、测试覆盖率、缺陷逃逸率、发布问题平均定位时间等。

3. 新增 AI 相关角色

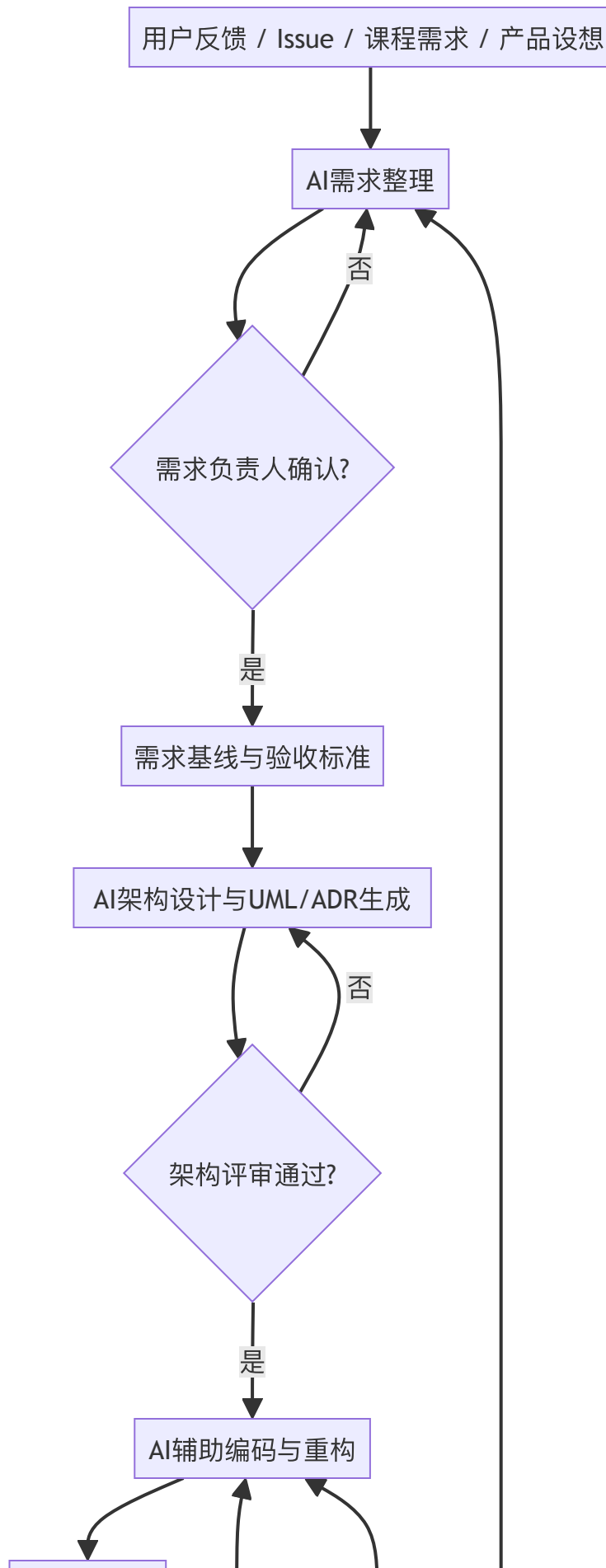
AI 工具进入流程后，需要有人维护提示词、审查输出、处理安全边界和复盘指标。小型团队可以一人兼任多个角色，但职责要写清楚。

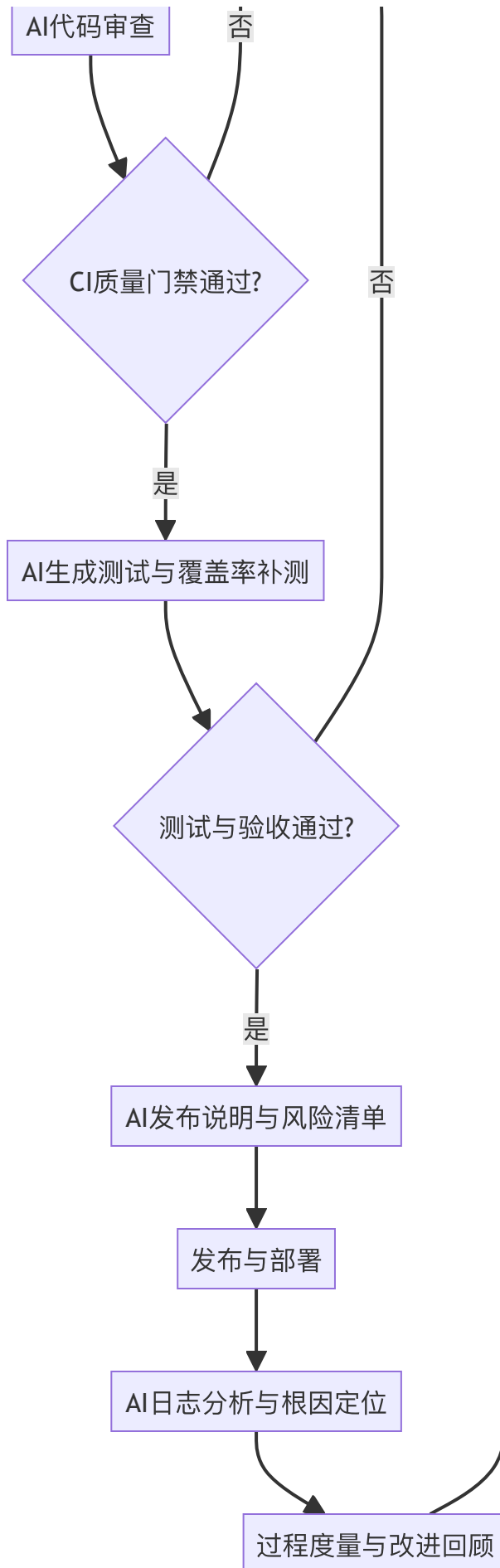
角色	核心职责	参与环节	主要输出
AI 需求分析师	使用 AI 整理访谈、issue、用户反馈，生成需求草案并检查一致性	需求分析	用户故事、需求规格、冲突清单、验收标准
AI 架构协同设计师	使用 AI 生成模块划分、UML、ADR 和设计模式建议	系统设计	架构草案、UML 图、ADR、架构评审清单
AI 代码协作者	操作 Codex、Claude Code、Cursor 等工具完成受控编码、重构和代码解释	编码实现	代码补丁、变更说明、重构建议
AI 测试分析师	使用 AI 生成测试用例、分析覆盖率缺口、辅助维护自动化测试	测试验证	测试用例、测试脚本、自愈建议、缺陷风险清单
AI 运维分析师	使用 AI 分析日志、错误栈、用户反馈和发布变更，定位可能根因	发布运维	告警摘要、根因候选、回滚建议、支持知识库条目
AI 安全与合规审查员	检查提示词注入、敏感数据泄露、依赖风险、权限越界等问题	全生命周期	安全审查记录、脱敏规则、风险处置建议

AI 过程负责人和 AI 安全与合规审查员偏治理，负责规则、记录和边界。AI 需求分析师、AI 架构协同设计师、AI 代码协作者、AI 测试分析师和 AI 运维分析师参与具体工作，产出会进入后续评审或质量门禁。

4. 总体流程重构

改进后的 Dyad 过程沿用原来的主线，只在关键节点加入 AI 生成、AI 检查、人工确认和自动验证。总体流程如下：



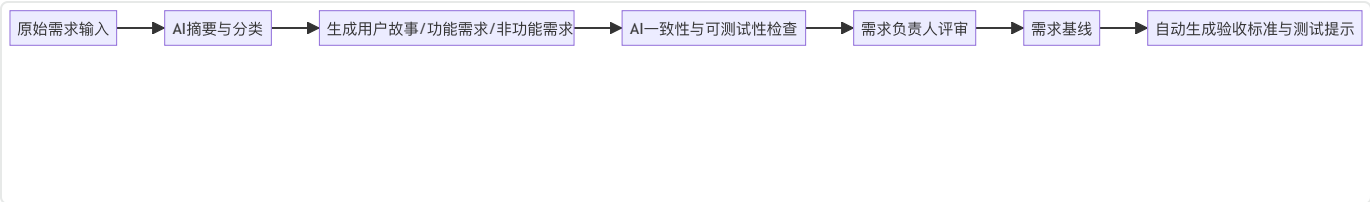


这条流程有三处回路：需求到设计用于澄清，实现到测试用于控质量，发布到复盘用于改进。AI 节点提供候选结果和分析结论，是否通过由负责人或自动化门禁决定。

5. 关键环节一：需求捕获与一致性验证改进

5.1 流程重构

传统流程中，需求通常来自课堂要求、用户描述、issue 和开发者讨论，后续整理主要靠人工归纳。改进后流程如下：



AI 需求分析师先把原始需求输入 LLM 或 RAG 系统，生成结构化草案，再让 AI 检查冲突、重复、范围膨胀和不可测试表达。例如，“完全本地运行”和“依赖云端模型”可能互相冲突；“界面体验好”也太宽泛，应拆成响应时间、交互步骤、错误提示、可访问性等可验证条件。

5.2 AI 输出物质量标准与验证机制

需求输出物包括用户画像、用户故事、功能需求、非功能需求、约束条件、验收标准和冲突清单。质量标准如下：

输出物	质量标准	验证机制
用户故事	包含角色、目标、收益，表达清晰	需求负责人逐条确认
功能需求	每条需求可追踪到来源，边界明确	与 issue、访谈或作业要求交叉检查
非功能需求	至少覆盖性能、稳定性、安全、兼容性、隐私	检查清单验证
验收标准	可测试、可判断、无含糊词	AI 测试分析师生成测试草案进行反向验证
冲突清单	标明冲突双方、原因和建议处理方式	需求评审会确认

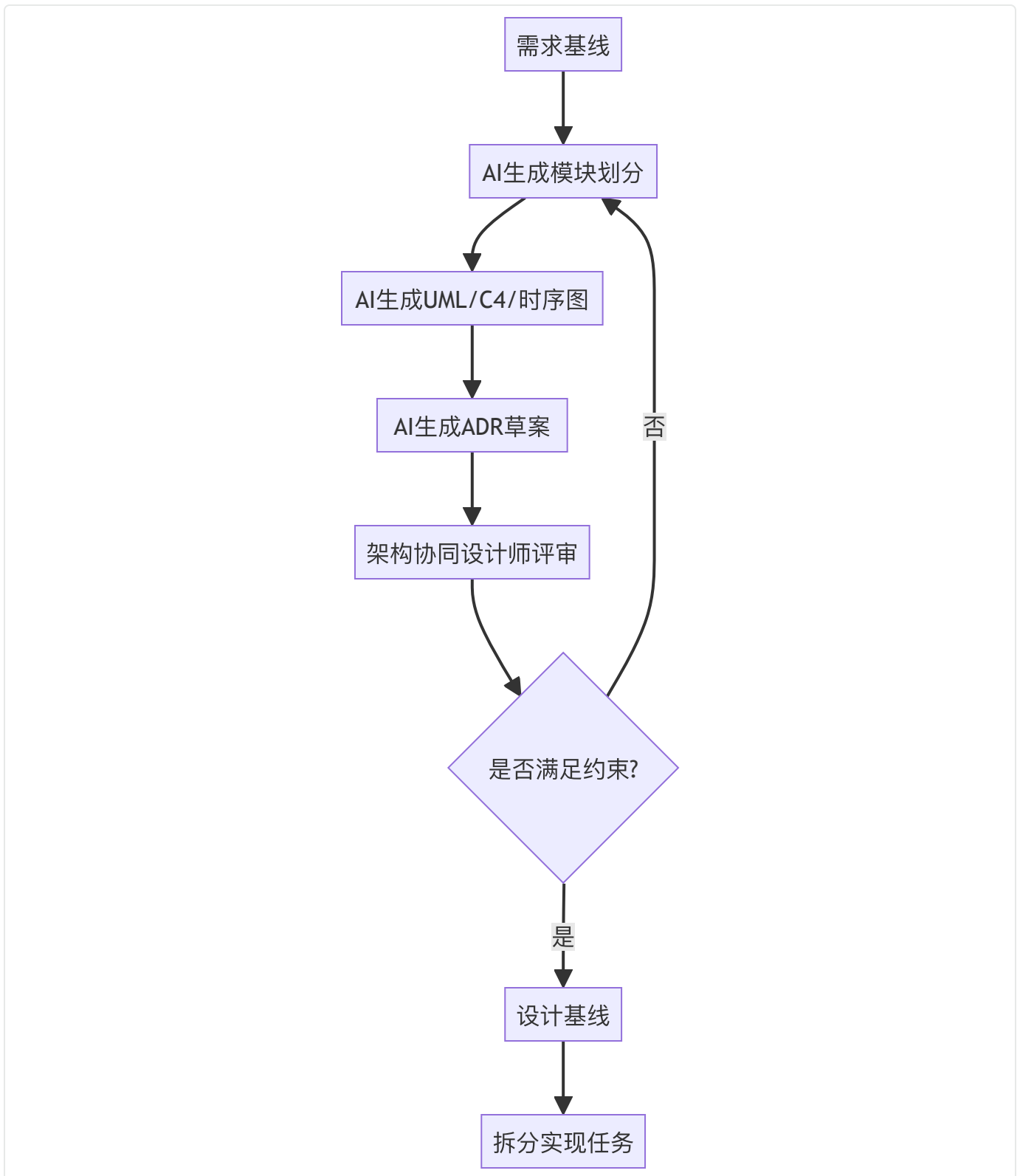
5.3 工具链选型

工具/框架	核心功能	集成方式	预期效率提升指标
Cursor	需求摘要、结构化规格生成、验收标准生成	接入 issue 模板或脚本，将 Markdown 需求输入模型后生成需求草案	需求初稿编写时间减少 40%—60%
GitHub Issues / GitLab Issues	需求来源管理、状态跟踪	在 issue 模板中增加 AI 摘要、验收标准和风险字段	需求可追溯率达到 90% 以上

6. 关键环节二：架构设计与 UML/ADR 生成改进

6.1 流程重构

传统架构设计依赖开发者经验，文档也容易落后于代码。改进后，把 AI 放进架构草拟和评审之间：



在 Dyad 中，AI 主要辅助划分 Electron 主进程、React 渲染进程、AI 对话服务、工程生成器、配置管理、项目预览、打包发布等模块边界。它还要写出关键数据流，例如“用户输入需求 -> 模型生成工程计划 -> 文件生成 -> 依赖安装 -> 本地预览 -> 错误反馈修复”。

6.2 AI 输出物质量标准与验证机制

架构输出物包括模块分解、接口草案、UML 图、ADR 和设计模式建议。质量标准如下：

输出物	质量标准	验证机制
模块分解	职责单一，边界清晰，不跨层访问	架构评审清单检查
UML/C4 图	与模块说明一致，关系方向明确	人工比对需求与代码结构
ADR	包含背景、决策、替代方案、后果	架构负责人签字确认
设计模式建议	说明适用原因和不用该模式的代价	原型验证或代码评审验证
接口草案	输入输出、错误码、权限边界明确	TypeScript 类型定义和契约测试验证

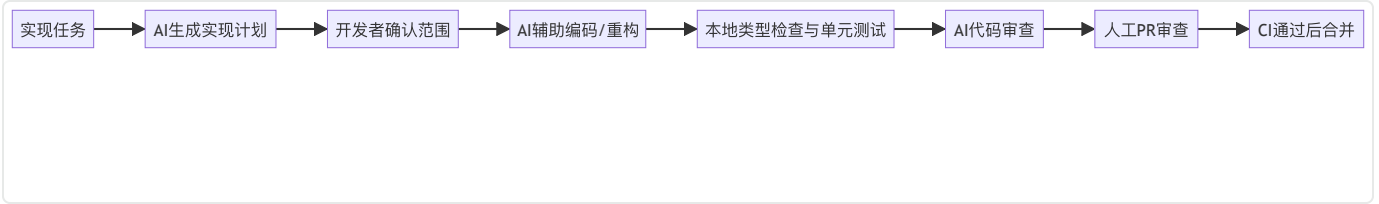
6.3 工具链选型

工具/框架	核心功能	集成方式	预期效率提升指标
Cursor	基于仓库上下文生成架构说明、模块边界和 ADR	在仓库中建立 docs/adr 、 docs/architecture ，由 Claude 读取上下文生成草案	架构文档起草时间减少 50%
Mermaid	将 AI 生成的结构化描述转换为 UML、流程图、时序图	在 Markdown 文档中嵌入 Mermaid	设计图维护成本降低 40%
Structurizr DSL	生成 C4 架构模型	将 AI 输出转为 DSL，纳入版本库	架构视图一致性提升

7. 关键环节三：编码实现与智能代码审查改进

7.1 流程重构

编码阶段主要改三件事：仓库级任务执行、受控重构和 PR 审查。流程如下：



AI 代码协作者不直接向核心分支提交代码，只在独立分支或工作区执行任务。每次 AI 变更都要附上变更说明、影响范围、测试结果和未解决问题。文件系统访问、模型调用、打包发布这类高风险模块，一次 AI 任务只处理一个明确问题。

7.2 AI 输出物质量标准与验证机制

输出物	质量标准	验证机制
实现计划	包含涉及文件、变更步骤、测试计划	开发者确认后执行
代码补丁	类型正确、风格一致、最小必要变更	TypeScript、ESLint、Prettier、单元测试
重构建议	不改变外部行为，说明收益和风险	快照测试、回归测试、人工 review
AI 代码审查报告	标明问题位置、严重程度、修复建议	PR 审查者确认
安全检查结果	覆盖输入校验、文件权限、提示词注入、依赖风险	安全清单和工具扫描

7.3 工具链选型

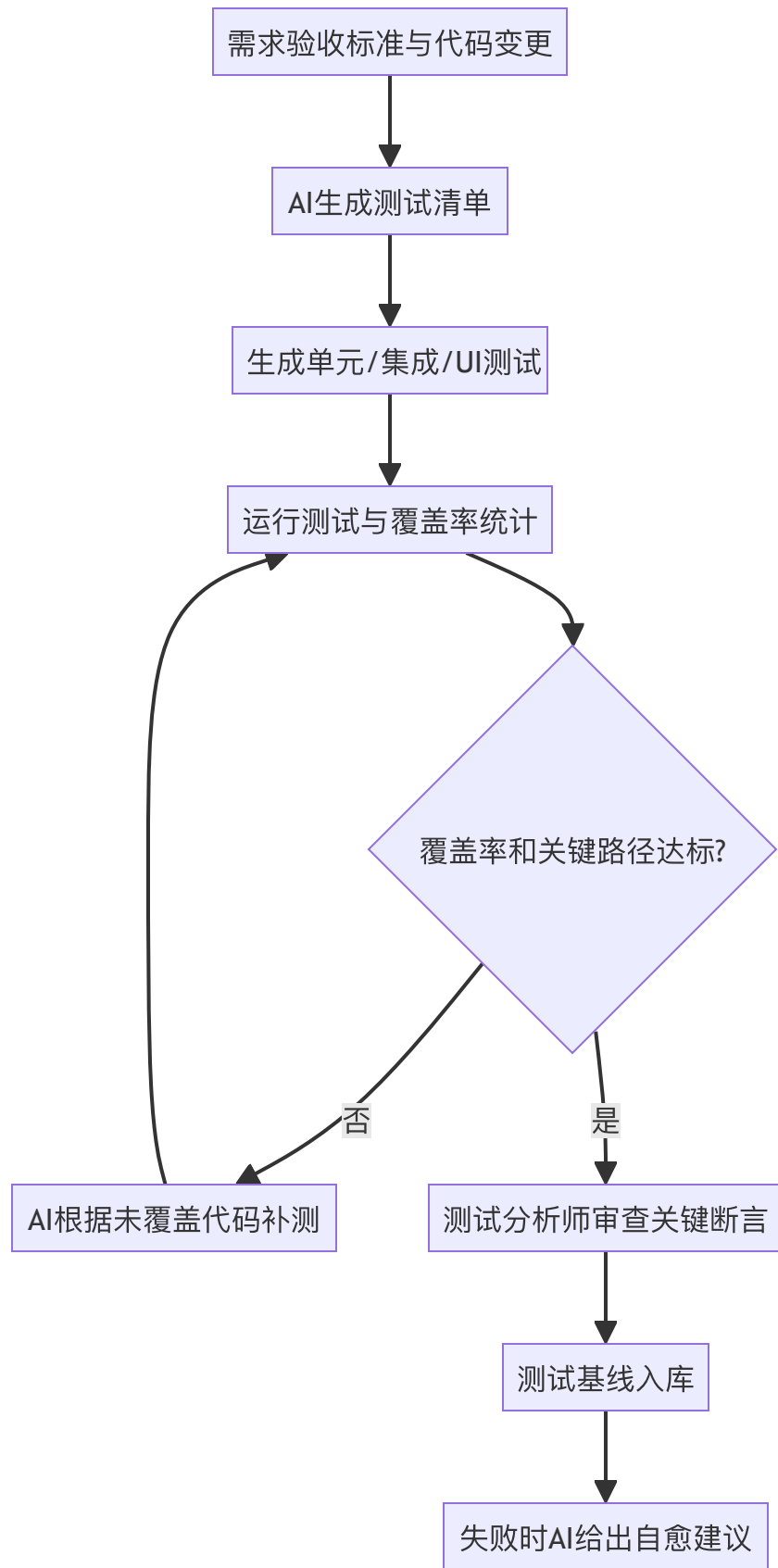
工具/框架	核心功能	集成方式	预期效率提升指标
Cursor /Bugbot	IDE 内补全、Agent 模式、PR 自动审查	开发者日常编码使用 Cursor，PR 阶段启用 Bugbot	重复性 review 问题发现效率提升 20%—40%
ESLint + TypeScript + Prettier	静态质量门禁	CI 必跑，AI 生成代码必须通过	风格与类型问题在合并前拦截

Snyk / Dependabot / npm audit	依赖漏洞和供应链风险 检查	PR 自动扫描依赖变更	依赖风险发现前移
----------------------------------	------------------	-------------	----------

8. 关键环节四：测试生成、自愈测试与缺陷预测改进

8.1 流程重构

测试阶段的重点是让 AI 根据需求、代码和覆盖率反馈持续补测，避免测试只在开发末尾一次性生成。流程如下：



Dyad 的测试重点包括工程生成器、提示词处理、配置保存、本地预览、跨平台路径、React 核心交互和 Electron 主进程安全边界。AI 测试分析师要检查测试断言是否真的覆盖业务行为，不能只盯实现细节。

8.2 AI 输出物质量标准与验证机制

输出物	质量标准	验证机制
测试清单	覆盖核心路径、异常路径、边界条件	对照需求验收标准
单元测试	断言明确，能稳定复现，不依赖外部不稳定服务	CI 多次运行
UI 测试	使用稳定选择器或可访问性定位	Playwright 测试执行
自愈建议	区分选择器失效和真实缺陷	测试分析师确认
缺陷风险评分	说明风险来源，如复杂度、变更范围、历史缺陷	PR 风险标签验证

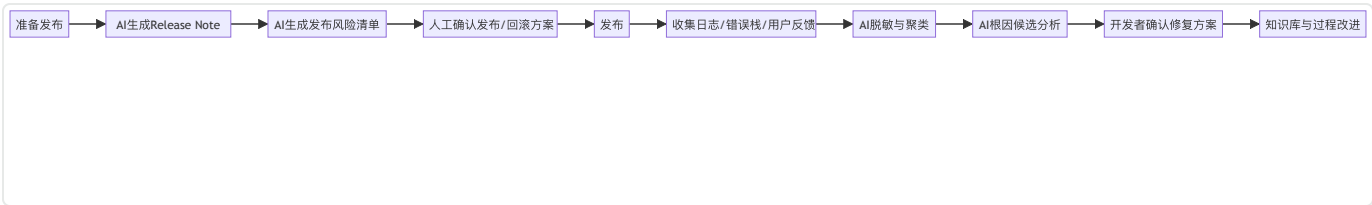
8.3 工具链选型

工具/框架	核心功能	集成方式	预期效率提升指标
Cursor	根据代码和需求生成单元测试、修复失败测试	在 PR 阶段自动生成测试建议，开发者确认后提交	测试编写时间减少 40%
Vitest / Jest	TypeScript 单元测试	AI 生成测试后由 CI 执行	单元测试覆盖率提升到 70% 以上
Chrome Dev Tools MCP	UI 自动化与跨浏览器测试	对核心用户路径建立 e2e 测试，失败时交给 AI 分析日志	核心流程回归时间减少 50%
Coverage 工具	覆盖率统计与缺口定位	CI 输出覆盖率报告，AI 根据未覆盖代码补测	未覆盖关键分支减少 30%
SonarQube / CodeQL	缺陷、复杂度和安全风险分析	PR 自动扫描，AI 汇总高风险项	高风险缺陷提前发现率提升 20%

9. 关键环节五：发布运维、异常检测与根因分析改进

9.1 流程重构

发布运维阶段主要让 AI 处理发布前风险清单、发布说明、日志聚类、根因候选和支持知识库。



对 Dyad 来说，这一环节重点分析安装包生成失败、依赖安装失败、本地预览失败、模型配置错误、权限不足、跨平台路径差异、用户输入导致生成异常等问题。AI 运维分析师只提供建议和脚本草案，不直接执行删除文件、回滚发布、修改用户数据等破坏性操作。

9.2 AI 输出物质量标准与验证机制

输出物	质量标准	验证机制
Release Note	覆盖新增、修复、破坏性变更、已知问题	与 commit、PR、issue 对照
发布风险清单	标明风险等级、影响用户、回滚方式	发布负责人确认
日志摘要	脱敏、聚类准确、保留关键错误栈	人工抽样检查
根因候选	给出证据链和置信度，不只给结论	开发者复现实验验证
支持知识库条目	包含现象、环境、原因、解决步骤	用户支持反馈验证

9.3 工具链选型

工具/框架	核心功能	集成方式	预期效率提升指标
Cursor	日志摘要、错误聚类、根因候选分析	将脱敏日志、错误栈、版本信息输入分析脚本	问题初步定位时间减少 40%

Sentry	前端与 Electron 异常采集	接入 Electron/React 错误上报，AI 汇总高频错误	发布后问题发现时间减少 30%
OpenTelemetry	指标、日志、链路追踪	为生成、预览、模型调用等关键路径埋点	性能瓶颈定位效率提升
GitHub Actions	发布流水线和自动化检查	发布前运行测试、构建、打包、AI 生成发布说明	发布准备时间减少 30%

10. 质量门禁与验证机制设计

AI 进入流程后，质量门禁要统一管理，避免模型输出绕过正常工程约束。

阶段	必过门禁	不通过时处理
需求阶段	需求来源可追踪、验收标准可测试、冲突已处理	返回 AI 需求整理器重新澄清
架构阶段	ADR 完整、模块边界清晰、接口可验证	重新生成方案或拆分原型验证任务
编码阶段	TypeScript、ESLint、单元测试、人工 review 通过	回到 AI 编码节点修复
测试阶段	核心路径测试通过、关键断言人工确认、覆盖率达标	AI 根据失败日志补测或修复
发布阶段	构建成功、发布说明完整、回滚方案明确	暂停发布
运维阶段	日志已脱敏、根因结论有证据、修复方案可复现	追加数据或人工排查

建议 Dyad 使用以下指标进行持续度量：

指标	基线获取方式	改进目标
需求初稿完成时间	统计每个需求从输入到规格草案的时间	降低 40%

PR 平均审查时间	GitHub PR 时间戳统计	降低 30%
测试覆盖率	CI 覆盖率报告	核心模块达到 70% 以上
缺陷逃逸率	发布后缺陷 / 总缺陷	降低 20%
平均问题定位时间	issue 创建到根因确认的时间	降低 40%
AI 输出采纳率	被接受的 AI 建议 / 总建议	稳定在 50%—70%，过高或过低都需复盘

11. 风险评估与应对策略

风险类型	具体表现	影响	应对策略
数据安全风险	将用户日志、API Key、商业需求、私有代码发送给外部模型	泄露隐私或敏感资产	建立脱敏脚本；敏感数据默认使用本地模型或企业版服务；禁止上传 <code>.env</code> 、密钥、用户原始数据
模型幻觉风险	AI 生成不存在的需求、错误架构、不可运行代码或虚假根因	造成返工和错误决策	所有 AI 输出进入评审；代码必须通过测试；根因必须有日志和复现实验证据
模型偏见与需求误读	AI 按常见产品假设补充需求，偏离 Dyad 的本地运行定位	需求范围膨胀	维护项目约束文档，提示词中固定“本地运行、轻量过程、MVP 优先”等原则
技术依赖风险	团队过度依赖某一商业 AI 工具，价格、服务稳定性或政策变化影响开发	过程不可持续	保留多工具方案；关键流程可切换 OpenAI、Claude、本地模型；核心文档和测试保存在版本库

质量责任不清	AI 生成代码后无人真正理解，出现问题难以追责	增加维护风险	PR 必须由人类负责人合并；AI 变更必须包含说明和测试；核心模块要求人工复述设计意图
安全边界风险	AI 生成文件操作、命令执行、依赖安装逻辑时引入漏洞	影响用户本地环境	对文件系统、shell、依赖安装、模型输出写文件设置安全审查；默认最小权限
自愈测试误判	AI 将真实缺陷当成测试脚本老化并自动修复	缺陷被掩盖	自愈建议必须人工确认；测试失败先分类为产品缺陷、测试缺陷或环境问题
成本失控	长上下文、多智能体和频繁调用导致费用上升	项目成本不可控	设置预算、缓存上下文、限制高成本模型调用；普通任务使用低成本模型
合规与版权风险	AI 生成代码可能受训练数据影响，引用不明来源实现	法律和许可风险	使用企业级工具条款；审查第三方代码片段；依赖必须明确许可证

12. 落地计划

本方案建议分三阶段实施。

第一阶段用 2—3 周建立低风险高收益的 AI 节点。交付物包括 AI 需求模板、需求一致性检查提示词、PR 审查提示词、Cursor 使用规范、Vitest 测试生成流程、发布说明生成模板。验收标准是：至少整理 3 个真实 issue，在 3 个 PR 上完成 AI 审查，并生成不少于 10 个可运行测试用例。

第二阶段用 4—6 周打通设计、实现、测试闭环。交付物包括架构 ADR 模板、Mermaid/UML 自动生成流程、核心模块测试覆盖率报告、Chrome Dev Tools MCP 核心流程测试、AI 测试补测流程。验收标准是工程生成器、配置管理、预览流程等核心模块覆盖率达到 70% 以上，至少 1 个设计决策有完整 ADR 和图示。

第三阶段用 6—8 周建立发布后反馈闭环。交付物包括发布风险清单、Release Note 自动生成、Sentry 或日志采集、日志脱敏脚本、根因分析模板、支持知识库。验收标准是发布后问题能在 24 小时内完成初

步分类，常见问题沉淀为知识库条目，并在迭代复盘形成过程改进建议。

13. 预期效果

实施后，Dyad 的软件过程会从人工驱动的轻量流程，变成带有 AI 验证节点的轻量流程。需求阶段更快形成结构化规格和验收标准；设计阶段保留必要的架构依据，不增加重文档负担；编码阶段用仓库级智能体和 AI 审查提高实现速度；测试阶段靠覆盖率反馈补测；发布运维阶段用日志摘要和根因候选缩短响应时间。

预期量化效果如下：需求初稿时间减少 40%—60%，架构文档起草时间减少约 50%，常规代码审查时间减少 30%—50%，测试用例编写时间减少约 40%，核心流程回归测试时间减少约 50%，发布后问题初步定位时间减少约 40%。这些指标不要求一次性全部达成，每轮迭代采集一次数据，再看哪些环节需要调整。

14. 结论

AI 技术进入软件过程后，真正要改的是活动、角色、输出物和验证机制。对 Dyad 这样的本地 AI 应用构建工具，更合适的做法是把 AI 放进需求、设计、编码、测试和运维的具体节点，再用质量门禁、人工确认、自动化测试和风险治理控制输出质量。该方案保留了 Dyad 原有最小可行过程的灵活性，也给高风险、高影响环节增加了可追溯控制，课程项目和实际开源项目都可以按阶段落地。

参考资料

[1] OpenAI, “Codex is now generally available,” 2025. <https://openai.com/index/codex-now-generally-available/>

[2] Anthropic, “Best practices for Claude Code,” 2025. <https://code.claude.com/docs/en/best-practices>

[3] Cursor, “Bugbot is now over 3x faster, 22% cheaper, and finds 10% more bugs,” 2026. <https://cursor.com/blog/bugbot-updates-june-2026>

[4] IBM, “AI coding agent for enterprises,” 2026. <https://www.ibm.com/products/ai-coding-agent>

[5] J. Yang et al., “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” NeurIPS, 2024. <https://arxiv.org/abs/2405.15793>

[6] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” ICLR, 2024. <https://openreview.net/forum?id=VTF8yNQM66>

